



# CSS DISPLAY TECHNIQUES

CSS Display Techniques & Comprehensive Layouts

# INTRODUCTION TO CSS DISPLAY

CSS display property controls how elements are rendered on a webpage.

It affects layout, alignment, and visibility.

# DISPLAY: BLOCK

Takes full width

Starts on a new line

Examples: `div`, `p`, `h1`

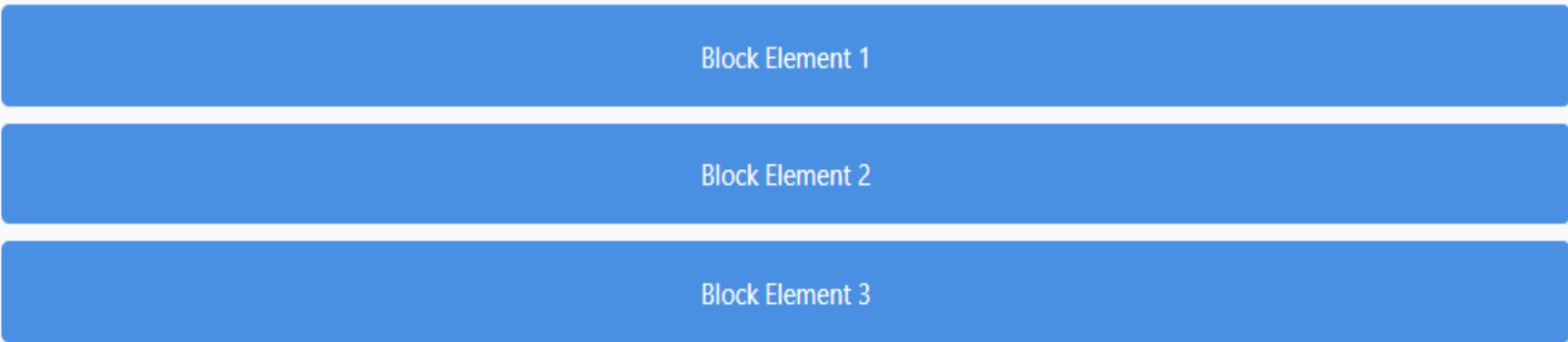
# Block Layout

---

**Block elements** take the full width available and stack vertically. They always start on a new line.

## display: block

These elements have display: block applied:



Block Element 1

Block Element 2

Block Element 3

```
/* CSS for block elements */ .block-item { display: block; /* Makes element a block */ background-color: #4a90e2; color: white; padding: 15px; margin: 10px 0; /* Vertical margin works */ width: 100%; /* Takes full width by default */ }
```

**Default Block Elements:** <div>, <h1>-<h6>, <p>, <section>, <article>

# DISPLAY: INLINE-BLOCK

Behaves like inline

Allows width & height

Useful for buttons and menus

# Inline-Block Layout

---

**Inline-block elements** flow horizontally like inline elements but can have width, height, margin, and padding like block elements.

## display: inline-block

These elements have display: inline-block applied:

Inline-Block 1

Inline-Block 2

Inline-Block 3

Inline-Block 4

```
/* CSS for inline-block elements */ .inline-block-item { display: inline-block; /* Allows horizontal flow with block
properties */ background-color: #34c759; color: white; padding: 15px; margin: 5px; /* All margins work */ width: 150px;
/* Width can be set */ height: 80px; /* Height can be set */ }
```

# DISPLAY: INLINE

Takes only required width

Does not start on new line

Examples: `span`, `a`

# BLOCK VS INLINE VS INLINE-BLOCK

## Comparison: Block vs Inline vs Inline-Block

### Block Elements

Takes full width

Stacks vertically

Accepts all box model properties

### Inline Elements

This is **Inline 1** text with **Inline 2**  
elements that flow **Inline 3** horizontally.

Inline elements ignore width/height and  
only have left/right margins

### Inline-Block Elements

Inline-Block  
1

Inline-Block  
2

Combines features of both: flows  
horizontally but accepts all box  
properties



# DISPLAY: NONE VS VISIBILITY: HIDDEN

`display:none` -removes element from layout

`visibility:hidden` -hides element but keeps space

# CSS FLOAT LAYOUT

Older layout method

Elements float left or right

Needs clear property

# CSS POSITIONING

## SYNTAX-"POSITION:STATIC"

### Types of CSS Positioning

There are 5 main values:

| Value                 | Meaning                         |
|-----------------------|---------------------------------|
| <code>static</code>   | Default position                |
| <code>relative</code> | Positioned relative to itself   |
| <code>absolute</code> | Positioned relative to parent   |
| <code>fixed</code>    | Positioned relative to viewport |
| <code>sticky</code>   | Mix of relative & fixed         |



# FLEXBOX LAYOUT

One-dimensional layout

Easy alignment & spacing

Used for navbars and cards

# FLEX LAYOUT

## Flexbox (Flexible Box Layout)

---

**Flexbox** is a one-dimensional layout method for arranging items in rows or columns.

### display: flex

This container has display: flex applied:

Flex Item 1

Flex Item 2

Flex Item 3

```
/* CSS for flex container */ .flex-container { display: flex; /* Activates Flexbox */ background-color: #e8f4fd; padding: 15px; gap: 10px; /* Space between items */ } .flex-item { flex: 1; /* Items grow equally to fill space */ padding: 20px; text-align: center; }
```

# CONTROLLING FLEX BOXES -JUSTIFY CONTENT

## Interactive Flexbox Controls

**justify-content:**

flex-start (default)

Items packed at the start of the container

**align-items:**

stretch (default)

Items stretch to fill the container (default)

Item 1 ssnvmncsm,dncmsdn,ms

Item 2 sncmsncmxsmcms cms

Item 3 sncms cmas ,ms m,as  
,snvms,msd,c m

```
/* Current Flexbox CSS */ #flexDemo { display: flex; justify-content: flex-start; align-items: stretch; }
```

# CONTROLLING FLEX BOXES -ALIGN ITEMS

## Interactive Flexbox Controls

**justify-content:**

flex-start (default)

Items packed at the start of the container

**align-items:**

center

Items centered vertically

Item 1 ssnvmncsmd,dncmsdn,ms

Item 2 sncmsncmxsmcms cms

Item 3 sncms cmas ,ms m,as  
,snvms,msd,c m

```
/* Current Flexbox CSS */ #flexDemo { display: flex; justify-content: flex-start; align-items: center; }
```

# CONTROLLING FLEX BOXES

## -ALIGN ITEMS

### Interactive Flexbox Controls

**justify-content:**

flex-start (default)

Items packed at the start of the container

**align-items:**

flex-start

Items aligned at the top of the container

Item 1 ssnvmncsmd,dncmsdn,ms

Item 2 sncmsncmxsmcms cms

Item 3 sncms cmas ,ms m,as  
,snvms,msd,c m

```
/* Current Flexbox CSS */ #flexDemo { display: flex; justify-content: flex-start; align-items: flex-start; }
```



# CONTROLLING FLEX BOXES -ALIGN ITEMS

## Interactive Flexbox Controls

**justify-content:**

flex-start (default)

Items packed at the start of the container

**align-items:**

flex-end

Items aligned at the bottom of the container

Item 1 ssnvmncsmd,dncmsdn,ms

Item 2 sncmsncmxsmcms cms

Item 3 sncms cmas ,ms m,as  
,snvms,msd,c m

```
/* Current Flexbox CSS */ #flexDemo { display: flex; justify-content: flex-start; align-items: flex-end; }
```

# GRID LAYOUT

Two-dimensional layout system

Rows and columns

Best for complex layouts

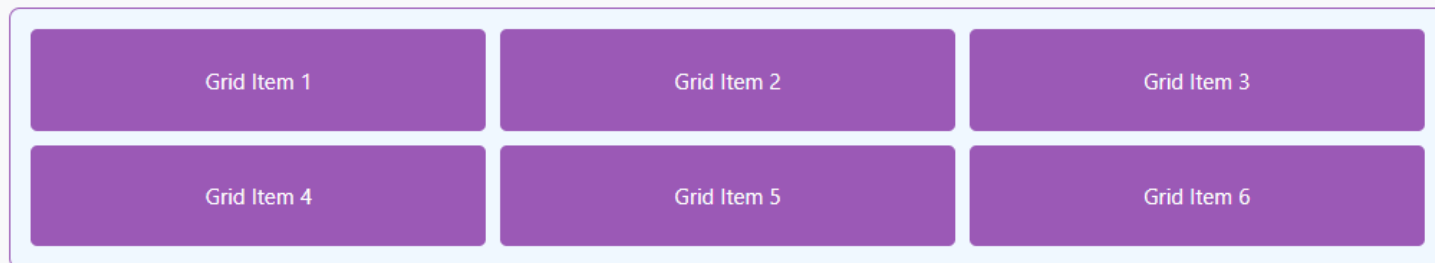
# GRID LAYOUT

## 4.4 CSS Grid Layout

**CSS Grid** is a two-dimensional layout system for creating complex grid-based designs.

### 4.4.1 display: grid

This container has display: grid applied:



```
/* CSS for grid container */ .grid-container { display: grid; /* Activates CSS Grid */ grid-template-columns: repeat(3, 1fr); /* Creates 3 equal columns */ gap: 10px; /* Space between grid items */ padding: 15px; } .grid-item { padding: 25px; text-align: center; }
```

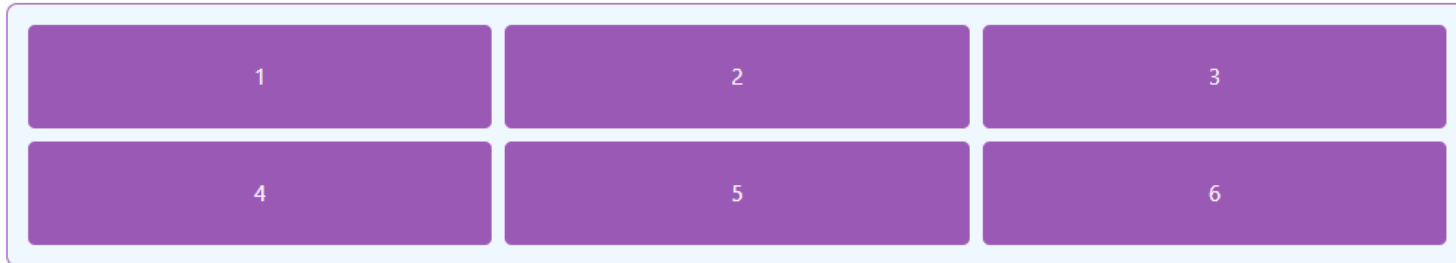
# CONTROLLING GRID BOXES -GRID TEMPLATE COLUMNS

## Interactive Grid Controls

### 4.4.2 grid-template-columns:

3 equal columns (1fr 1fr 1fr)

3 equal columns that fractionally divide available space



```
/* Current Grid CSS */ #gridDemo { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; }
```

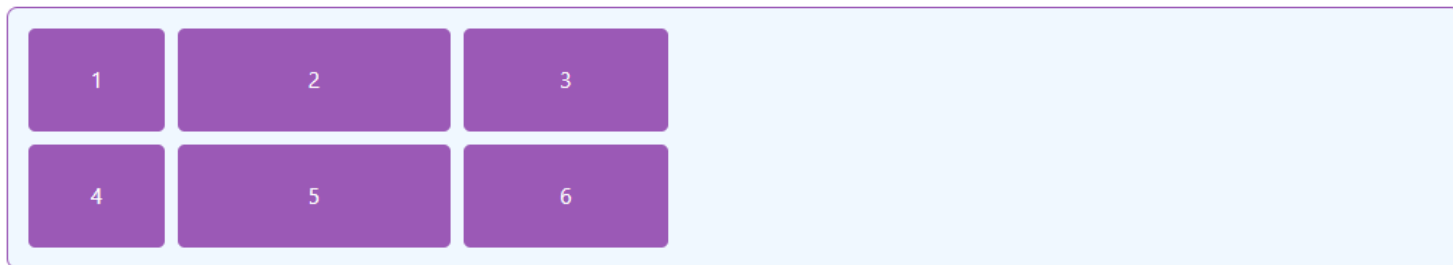
# CONTROLLING GRID BOXES -GRID TEMPLATE COLUMNS

## Interactive Grid Controls

### 4.4.2 grid-template-columns:

Fixed widths (100px 200px 150px)

3 columns with fixed pixel widths



```
/* Current Grid CSS */ #gridDemo { display: grid; grid-template-columns: 100px 200px 150px; gap: 10px; }
```

# CONTROLLING GRID BOXES

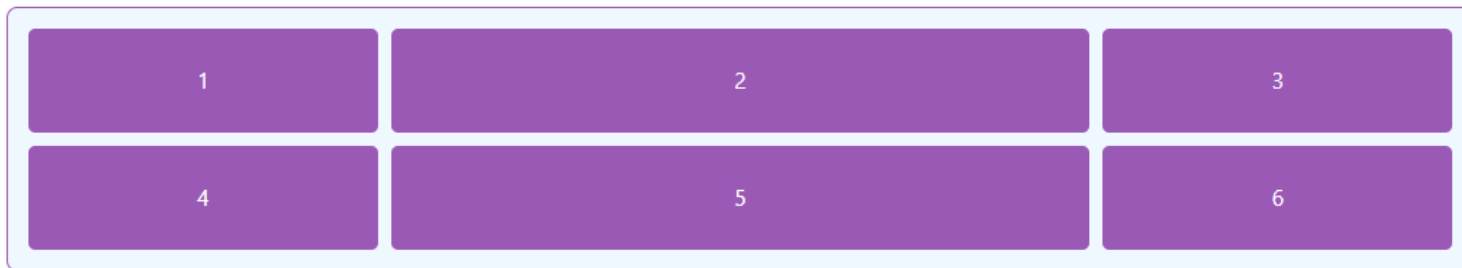
## -GRID TEMPLATE COLUMNS

### Interactive Grid Controls

#### 4.4.2 grid-template-columns:

Mixed ratio (1fr 2fr 1fr)

3 columns with middle column twice as wide as others



```
/* Current Grid CSS */ #gridDemo { display: grid; grid-template-columns: 1fr 2fr 1fr; gap: 10px; }
```

# SUMMARY

## Layout Properties Summary

| Property              | Purpose                              | Example Values                          |
|-----------------------|--------------------------------------|-----------------------------------------|
| display               | Defines layout behavior              | block, inline, inline-block, flex, grid |
| justify-content       | Aligns items on main axis (Flexbox)  | flex-start, center, space-between       |
| align-items           | Aligns items on cross axis (Flexbox) | stretch, center, flex-start             |
| grid-template-columns | Defines grid columns                 | repeat(3, 1fr), 100px 200px, 1fr 2fr    |
| gap                   | Space between items                  | 10px, 1rem, 5%                          |

# CONCLUSION

Modern layouts prefer Flexbox & Grid

Display property is foundation of CSS layouts



# SAMPLE CODE

```
<> csslayouts_ds.html > html > head > style > header
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Chapter: Comprehensive CSS Layouts</title>
7  <style>
8  /* =====
9  | GLOBAL STYLES & RESET
10 | These styles apply to the entire page
11 | ===== */
12 * {
13   box-sizing: border-box; /* Makes padding and border included in element's width/height */
14   margin: 0;
15   padding: 0;
16   font-family: 'Segoe UI', Arial, sans-serif;
17 }
18
19 body {
20   background-color: #f5f7fa;
21   color: #333;
22   line-height: 1.6;
23   padding: 20px;
24   max-width: 1200px;
25   margin: 0 auto;
26 }
27
28 /* Header Styles */
29 header {
30   background: linear-gradient(135deg, #1e3c72, #2a5298);
31   color: white;
32   padding: 30px;
33   text-align: center;
34   border-radius: 10px;
35   margin-bottom: 30px;
36   box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
37 }
```

```
h1 {
  font-size: 2.5rem;
  margin-bottom: 10px;
}

h2 {
  color: #1e3c72;
  margin: 25px 0 15px;
  padding-bottom: 8px;
  border-bottom: 3px solid #1e3c72;
}

h3 {
  color: #2a5298;
  margin: 20px 0 10px;
}

label {
  display: inline-block;
  width: 200px;
  font-weight: bold;
  color: #1e3c72;
}

select {
  padding: 8px 15px;
  border: 2px solid #ddd;
  border-radius: 5px;
  font-size: 1rem;
  width: 250px;
}
```

```
/* Footer */
footer {
  text-align: center;
  margin-top: 40px;
  padding: 20px;
  color: #666;
  border-top: 1px solid #e1e5eb;
}

/* Container for each section */
.section {
  background-color: white;
  border-radius: 8px;
  padding: 25px;
  margin-bottom: 30px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.08);
}

.example-box {
  border: 2px solid #e1e5eb;
  border-radius: 8px;
  padding: 20px;
  margin: 15px 0;
  background-color: #f8f9fa;
}
```

```
.code-block {
  background-color: #2d3748;
  color: #e2e8f0;
  padding: 15px;
  border-radius: 5px;
  font-family: 'Courier New', monospace;
  margin: 10px 0;
  overflow-x: auto;
  font-size: 0.9rem;
}

.note {
  background-color: #e8f4fd;
  border-left: 4px solid #2a5298;
  padding: 12px 15px;
  margin: 15px 0;
  border-radius: 0 5px 5px 0;
}
```

```
/* =====  
BLOCK LAYOUT  
Elements that take full width and stack vertically  
===== */  
.block-container {  
border: 2px dashed #ccc;  
padding: 15px;  
margin: 15px 0;  
}  
  
.block-item {  
display: block; /* This is the default for div, h1-h6, p, etc. */  
background-color: #4a90e2;  
color: white;  
padding: 15px;  
margin: 10px 0;  
border-radius: 5px;  
text-align: center;  
}
```

```
/* =====  
  INLINE-BLOCK LAYOUT  
  Elements that sit side-by-side but can have width/height  
===== */  
.inline-block-container {  
  border: 2px dashed #ccc;  
  padding: 15px;  
  margin: 15px 0;  
}  
  
.inline-block-item {  
  display: inline-block; /* Allows elements to sit side-by-side */  
  background-color: #34c759;  
  color: white;  
  padding: 15px;  
  margin: 5px;  
  border-radius: 5px;  
  width: 150px; /* Inline-block elements CAN have width */  
  height: 80px; /* Inline-block elements CAN have height */  
  text-align: center;  
  vertical-align: top; /* Aligns elements at the top */  
}  
  
.inline-example {  
  display: inline; /* Inline elements ignore width/height */  
  background-color: #ff9500;  
  color: white;  
  padding: 5px 10px;  
  margin: 0 5px;  
}
```

```

/* =====
FLEXBOX LAYOUT
One-dimensional layout system
===== */

.flex-container {
  display: flex; /* Activates Flexbox */
  background-color: #e8f4fd;
  border: 2px solid #4a90e2;
  border-radius: 8px;
  padding: 15px;
  margin: 15px 0;
  min-height: 200px;
}

.flex-item {
  background-color: #4a90e2;
  color: white;
  padding: 20px;
  margin: 5px;
  border-radius: 5px;
  text-align: center;
  flex: 1; /* Allows items to grow and shrink equally */
  justify-content: center;
  /*other options: flex-start; (Default - items at the start), center; (Items centered along main axis) flex-end; (Items at the end) */
  /*space-between; (Space between items) space-around; (Space around items)*/
  align-items: center;
  /*other options: flex-start; (Default - Items aligned at the top), center; (Items centered vertically) flex-end; (Items aligned at the bottom) */
  /*stretch; (Items stretch to fill container (default))*/
}

```

```
/* =====
CSS GRID LAYOUT
Two-dimensional layout system
===== */

.grid-container {
  display: grid; /* Activates CSS Grid */
  background-color: #f0f8ff;
  border: 2px solid #9b59b6;
  border-radius: 8px;
  padding: 15px;
  margin: 15px 0;
  gap: 10px; /* Space between grid items */
}

.grid-item {
  background-color: #9b59b6;
  color: white;
  padding: 25px;
  border-radius: 5px;
  text-align: center;
}

.grid-cols{
  grid-template-columns: repeat(2, 1fr); /* 3 equal columns */
}

.grid-rows{
  grid-template-rows: repeat(2,1fr);/* 3 equal rows */
}

</style>
</head>
```



```
<body>
<header>
  <h1>Chapter: Comprehensive CSS Layouts</h1>
  <p>For First Year Engineering Students</p>
</header>

<main>
  <section class="section">
    <h2>Block Layout</h2>
    <p><strong>Block elements</strong> take the full width available and stack vertically. They always start on a new line.</p>

    <div class="example-box">
      <h3>display: block</h3>
      <p>These elements have <code>display: block</code> applied:</p>

      <div class="block-container">
        <div class="block-item">Block Element 1</div>
        <div class="block-item">Block Element 2</div>
        <div class="block-item">Block Element 3</div>
      </div>

      <div class="code-block">
/* CSS for block elements */
.block-item {
display: block;           /* Makes element a block */
background-color: #4a90e2;
color: white;
padding: 15px;
margin: 10px 0;          /* Vertical margin works */
width: 100%;             /* Takes full width by default */
}
      </div>
    </section>
  </main>
</body>
```

```
<div class="note">
  <strong>Default Block Elements:</strong> &lt;div&gt;, &lt;h1&gt;,&lt;h6&gt;, &lt;p&gt;, &lt;section&gt;, &lt;article&gt;
</div>
</div>
</section>

<section class="section">
<h2>Inline-Block Layout</h2>
<p><strong>Inline-block elements</strong> flow horizontally like inline elements but can have width, height, margin, and padding like block elem

<div class="example-box">
  <h3>display: inline-block</h3>
  <p>These elements have <code>display: inline-block</code> applied:</p>

  <div class="inline-block-container">
    <div class="inline-block-item">Inline-Block 1</div>
    <div class="inline-block-item">Inline-Block 2</div>
    <div class="inline-block-item">Inline-Block 3</div>
    <div class="inline-block-item">Inline-Block 4</div>
  </div>

  <div class="comparison-box">
    <h4>Inline Elements</h4>
    <p>This is <span class="inline-example">Inline 1</span> text with
    <span class="inline-example">Inline 2</span> elements that flow
    <span class="inline-example">Inline 3</span> horizontally.</p>
    <div class="note">Inline elements ignore width/height and only have left/right margins</div>
  </div>
</div>
```

```
<section class="section">
  <h2>Flexbox (Flexible Box Layout)</h2>
  <p><strong>Flexbox</strong> is a one-dimensional layout method for arranging items in rows or columns.</p>

  <div class="example-box">
    <h3>display: flex</h3>
    <p>This container has <code>display: flex</code> applied:</p>

    <div class="flex-container">
      <div class="flex-item">Flex Item 1</div>
      <div class="flex-item">Flex Item 2</div>
      <div class="flex-item">Flex Item 3</div>
    </div>

    <div class="code-block">
      .flex-item {

        flex: 1; /* Allows items to grow and shrink equally */
        justify-content: center;
        /*other options: flex-start; (Default - items at the start), center; (Items centered along main axis) flex-end; (Items at the end) */
        /*space-between; (Space between items) space-around; (Space around items)*/

        align-items: center;
        /*other options: flex-start; (Default - Items aligned at the top), center; (Items centered vertically) flex-end; (Items aligned at the bottom) */
        /*stretch; (Items stretch to fill container (default))*/
      }

    </div>
  </div>
</section>
```

```
<section class="section">
  <h2>CSS Grid Layout</h2>
  <p><strong>CSS Grid</strong> is a two-dimensional layout system for creating complex grid-based designs.</p>

  <div class="example-box">
    <h3>display: grid</h3>
    <p>This container has <code>display: grid</code> applied:</p>

    <div class="grid-container grid-cols">
      <div class="grid-item">Grid Item 1</div>
      <div class="grid-item">Grid Item 2</div>
      <div class="grid-item">Grid Item 3</div>
      <div class="grid-item">Grid Item 4</div>
      <div class="grid-item">Grid Item 5</div>
      <div class="grid-item">Grid Item 6</div>
    </div>

    <div class="code-block">
/* CSS for grid container */

/* Different grid-template-columns examples */
.grid-cols-3 {
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
}

.grid-cols-fixed {
  grid-template-columns: 100px 200px 150px; /* Fixed width columns */
}
```

```
</section>
</main>
```

```
<footer>
  <p>Comprehensive CSS Layouts | For First Year Engineering Students</p>
  <p>Resize browser window to see responsive behavior</p>
</footer>
```

```
</script>
</body>
</html>
```